

Chapter 1: Introduction

1.1 Introduction

This report presents the verification and validation methodology adopted for a 32-bit, five-stage pipelined RISC-V processor implementing the RV32I base integer instruction set. The RV32I ISA was selected due to its simplicity, open-source architecture, and suitability for low-power embedded and IoT applications. The processor incorporates the standard IF, ID, EX, MEM, and WB pipeline stages, along with forwarding, hazard detection, and pipeline flush mechanisms to ensure correct execution of instructions.

Since the individual RTL modules had already been verified independently, the primary objective of this work was to validate the complete integrated processor. The verification methodology followed a systematic progression, beginning with manual instruction-level verification, followed by pipeline hazard validation, software execution using the GNU RISC-V toolchain, architectural comparison with the Spike ISA simulator, and finally constrained-random verification using the RISC-V DV framework. This phased approach enabled thorough debugging and validation of the processor under increasingly complex execution scenarios.

1.2 Processor Features

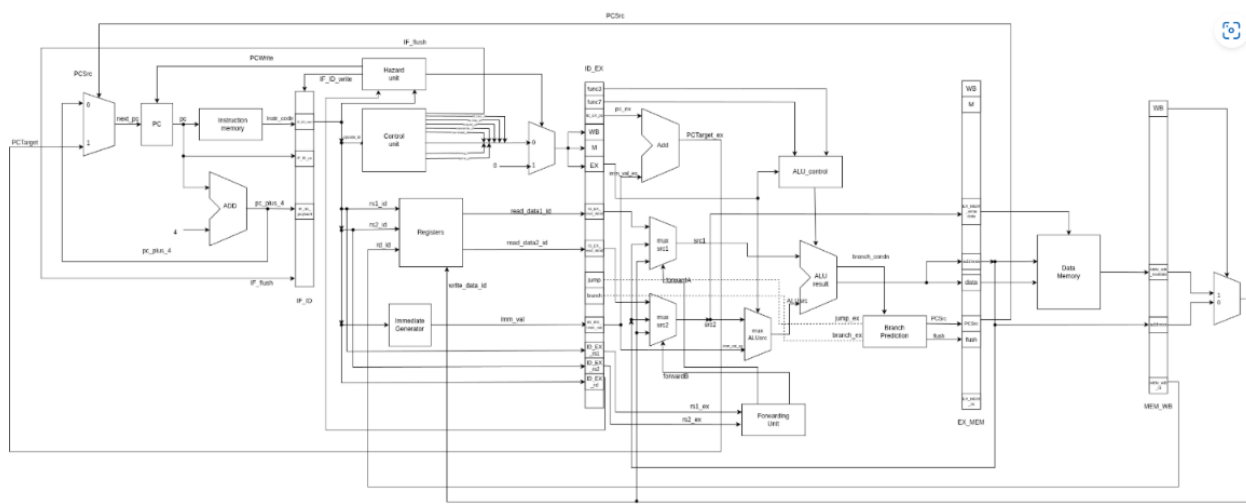
Feature	Description
ISA	RV32I Base Integer ISA
Architecture	32-bit RISC-V Processor
Pipeline	Five-stage in-order pipeline
Pipeline Stages	IF, ID, EX, MEM, WB
Register File	32 × 32-bit General Purpose Registers
Instruction Memory	4 KB
Data Memory	16 KB
Hazard Handling	Data Forwarding and Hazard Detection Unit
Control Hazard Handling	Branch Flush Mechanism
Branch Support	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump Support	JAL, JALR
Verification Methodology	Directed Testing, GCC Programs, Spike Comparison, RISC-V DV

1.3 Verification Philosophy

This is where your report becomes different from the other team's.

Since all individual RTL modules had already undergone independent unit-level verification, the focus of this work was shifted toward **processor-level validation**. The primary objective was not merely to verify isolated functional blocks, but to ensure that the integrated processor correctly executed software programs while maintaining architectural compliance with the RV32I specification. The verification methodology was therefore organized as a sequence of progressively complex validation phases. The verification process began with manual instruction-level testing using hand-generated machine code, followed by directed verification of pipeline hazards, execution of C programs through the GNU RISC-V toolchain, architectural comparison against the Spike ISA simulator, and finally constrained-random verification using the RISC-V DV framework. This incremental approach enabled systematic debugging and validation of the processor while gradually increasing the complexity of executed workloads.

Figure 1.1: Top-Level Architecture of the Five-Stage RV32I Processor.



Chapter 2: Verification Methodology

2.1 Verification Strategy

The objective of the verification process was to ensure that the integrated RV32I processor executed instructions correctly while maintaining architectural compliance with the RISC-V ISA specification. Since all individual RTL modules had already undergone unit-level verification, this work focused on validating the complete processor through a structured, phase-wise verification methodology.

Rather than directly executing complex software applications, the verification process progressed from simple directed testing to advanced architectural validation. Each phase increased the complexity of the executed programs while simultaneously validating additional processor functionality. This incremental approach enabled efficient debugging of the processor and ensured that issues identified at each stage were resolved before proceeding to the next level of verification.

The complete verification methodology adopted during this work is summarized in Flow graph below.

Manual Instruction Verification

|



Instruction-Level Debugging (Waveform Analysis)

|



Pipeline Hazard Verification (Data & Control Hazards)

|



GCC Toolchain Setup (C Program Execution)

|



Spike ISA Simulation (Golden Reference)

|



Long Program Verification

|



RISC-V DV Setup

|



Constrained Random Verification

2.2 Verification Phases

The complete processor verification was divided into five major phases.

Phase I – Manual Instruction Verification:

Individual RV32I instructions were manually encoded into machine code and loaded into the instruction memory. The execution of each instruction was verified using simulation waveforms and expected register or memory values. This phase established the functional correctness of all supported instructions.

Phase II – Pipeline Hazard Verification:

After validating individual instructions, directed programs were developed to verify processor pipeline behavior under different hazard conditions. Data hazards, forwarding paths, load-use hazards, branch instructions, and control hazards were systematically tested and debugged.

Phase III – Software Execution using the GNU Toolchain:

Following successful directed verification, the GNU RISC-V toolchain was integrated into the verification flow. C programs were compiled into executable binaries, converted into instruction memory images, and executed on the processor to validate complete software execution.

Phase IV – Architectural Validation using Spike:

The Spike ISA simulator was used as the architectural golden reference. Execution logs generated from Spike were compared against RTL execution logs using an automated Python-based comparison framework, enabling instruction-by-instruction verification of processor behavior.

Phase V – Constrained-Random Verification:

Finally, the RISC-V DV framework was integrated to generate randomized instruction sequences for stress testing the processor. Random program execution further increased verification coverage by exercising complex instruction combinations and corner-case scenarios beyond manually developed test programs.

Chapter 3: Phase I – Manual Instruction-Level Verification

3.1 Verification Procedure

1. Write Assembly
↓
2. Generate Hex
↓
3. Put Hex into imem.mem
↓
4. Simulate RTL
↓
5. Observe Waveforms
↓
6. Compare Expected Register Values
↓
7. Debug if Required
↓
8. Re-run until PASS

3.2 Instruction Categories

Category	Instructions Verified
Arithmetic	ADD, SUB
Logical	AND, OR, XOR
Shift	SLL, SRL, SRA
Immediate	ADDI, ANDI, ORI, XORI, SLTI, SLTIU
Comparison	SLT, SLTU
Upper Immediate	LUI, AUIPC
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump	JAL, JALR
Memory	LB, LH, LW, LBU, LHU, SB, SH, SW

3.4 Example Instruction Verification

For example:

```
addi x1,x0,10
addi x2,x0,20
add x3,x1,x2
```

Expected: $x3 = 30$

Venus Editor Simulator Chocopy

Run Step Prev Reset Dump Trace Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x00A00093	addi x1 x0 10	addi x1,x0,10
0x4	0x01400113	addi x2 x0 20	addi x2,x0,20
0x8	0x002081B3	add x3 x1 x2	add x3,x1,x2

Copy! Download! Clear!

0x00A00093
0x01400113
0x002081B3

Figure 3.1: Machine Code Generation for ADD Instruction

Generation of hexadecimal machine code for the directed ADD instruction test using the Venus RISC-V assembler. The handwritten assembly program was assembled into its corresponding 32-bit machine code, which was then loaded into the processor instruction memory (`imem.mem`) for RTL simulation.

Hex-Codes for given assembly:

0x00A00093

0x01400113

0x002081B3

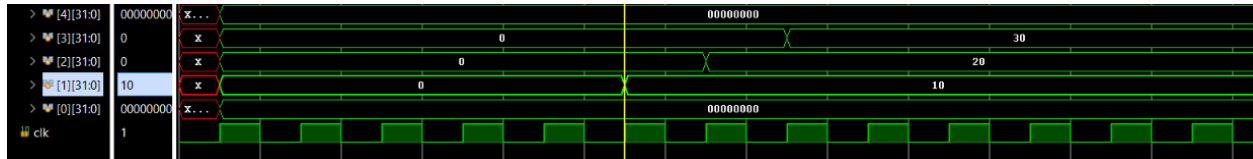


Figure 3.2: RTL Verification of ADD Instruction

It illustrates the RTL simulation waveform for the directed ADD instruction test. Initially, the processor executes `addi x1, x0, 10`, updating register **x1** with the value **10**. The next instruction, `addi x2, x0, 20`, writes the value **20** into register **x2**. Finally, the instruction `add x3, x1, x2` computes the sum of the two source registers and writes the expected result **30** into register **x3**. The observed register updates matched the expected behavior specified by the RV32I ISA, confirming the correct implementation of arithmetic instruction execution and register write-back.

Assembly Instruction	Machine Code	Expected Result
<code>addi x1, x0, 10</code>	00A00093	x1 = 10
<code>addi x2, x0, 20</code>	01400113	x2 = 20
<code>add x3, x1, x2</code>	002081B3	x3 = 30

3.5 Outcome of Phase I

After completing manual instruction-level verification, all supported RV32I instruction categories executed according to the ISA specification. The processor datapath, control logic, register file, immediate generation, ALU operations, memory interface, and write-back path were validated through directed testing. This established a stable baseline for progressing to pipeline hazard verification, where interactions between multiple instructions could be analyzed under realistic execution conditions.

Chapter 4: Phase II – Pipeline Hazard Verification

4.1 Introduction

After successful verification of individual RV32I instructions, the next stage focused on validating the pipelined execution of the processor. A set of directed assembly programs was developed to intentionally generate different pipeline hazards and verify the correctness of the forwarding, hazard detection, stall generation, and pipeline flush mechanisms. The processor behavior was analyzed using RTL simulation waveforms, and any observed issues were debugged and resolved before proceeding to software-based verification.

4.2 Hazard Verification

The following pipeline hazards were verified during this phase:

Data Hazards

- Read After Write (RAW) Hazard
- EX/MEM Forwarding
- MEM/WB Forwarding
- Load-Use Hazard
- Consecutive Data Hazards
- Double Load-Use Hazard

Control Hazards

- BEQ
- BNE
- BLT
- BGE
- JAL
- JALR

All hazard scenarios were verified using directed assembly programs and validated through RTL simulation waveforms.

4.3 Representative Hazard Verification

The following directed program was used to verify the Load-Use Hazard.

```
lw    x5, 0(x10)
add   x6, x5, x7
```

Since the loaded data is not immediately available in the next pipeline stage, the Hazard Detection Unit identifies the dependency and inserts a single pipeline stall before executing the dependent instruction. The generated waveform confirms that the pipeline is stalled for one cycle, after which the correct value is forwarded and normal execution resumes.

```

addi x10, x0, 0      # Base address of data memory
addi x7,  x0,  5      # x7 = 5

lw   x5, 0(x10)      # Load word from memory
add  x6, x5, x7       # Dependent instruction (Load-Use Hazard)

addi x8, x6, 1        # Verify correct forwarding after stall

```

Register	Expected Value
x5	10
x6	15
x8	16

Figure 4.1: Load-Use Hazard verification waveform.

4.4 Debugging During Hazard Verification

During hazard verification, a functional issue was observed while testing the Double Load-Use Hazard scenario. The processor produced incorrect results because the Register File write operation was occurring before the read operation within the same clock cycle.

To resolve this issue, the Register File access priority was modified so that register reads occur before writes during the clock cycle. After implementing this change, the Double Load-Use Hazard executed correctly, and the expected register values were observed during simulation.

4.5 Verification Summary

Hazard Type	Status
RAW Hazard	PASS
EX/MEM Forwarding	PASS
MEM/WB Forwarding	PASS
Load-Use Hazard	PASS
Consecutive Data Hazard	PASS
Double Load-Use Hazard	PASS

BEQ/BNE	PASS
BLT/BGE	PASS
JAL/JALR	PASS

Chapter 5: Phase III – Software Execution using the GNU RISC-V Toolchain

5.1 Introduction

After successful instruction-level and pipeline hazard verification, the processor was validated using complete software programs written in the C programming language. Executing C programs significantly increases the complexity of verification by exercising multiple instruction types, control flow, loops, arithmetic operations, memory accesses, and data dependencies within a single program.

To achieve this, the GNU RISC-V Toolchain was integrated into the verification flow. The toolchain automatically converts C source code into RV32I executable machine code, eliminating the need for manually encoding instructions. The generated executable was then converted into a hexadecimal memory image and loaded into the processor instruction memory for RTL simulation.

This phase verified the processor's ability to execute complete software programs while ensuring correct instruction execution, memory access, register updates, and overall program behavior.

5.2 GNU Toolchain Verification Flow

C Program

|



RISC-V GCC Compiler

|



ELF Executable

|



Objcopy

|



Binary File

|



Hex Memory Image (imem.mem)

|



RTL Simulation

|



Waveform & Testbench Verification

5.3 Build Flow & Commands

Step 1: Compile the C Program

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -O2 -T gcc_files/link.ld -nostdlib -nostartfiles  
gcc_files/crt0.S gcc_files/tst.c -o gcc_files/tst.elf
```

Step 2: Inspect the Executable

```
riscv64-unknown-elf-objdump -d gcc_files/tst.elf
```

Step 3: View Symbol Addresses

```
riscv64-unknown-elf-nm gcc_files/tst.elf
```

Step 4: Convert ELF to Binary

```
riscv64-unknown-elf-objcopy -O binary gcc_files/tst.elf test.bin
```

Step 5: Generate Instruction Memory

```
hexdump -v -e '1/4 "%08X\n"' test.bin >> ~/Desktop/processor/verilator/imem.mem
```

Step 5: Generate Data Memory

```
riscv64-unknown-elf-objcopy -O binary --only-section=.data test.elf dmem.bin
```

```
hexdump -v -e '1/4 "%02X\n"' dmem.bin > dmem.mem
```

Step 6: Run RTL Simulation

```
./run.sh
```

5.4 Software Verification

Multiple C programs were executed to validate different processor functionalities.

Program	Purpose	Status
Arithmetic Operations	ALU Verification	PASS
Memory Access	Load/Store Verification	PASS
Branch Program	Control Flow Verification	PASS
Loop Program	Loop Execution Verification	PASS
Function Call	JAL/JALR Verification	PASS

5.5 Example Verification

5.5.1 C Program

```
int main()
```

```
{
```

```

int a = 10;

int b = 20;

int c = a + b;

return c;

}

```

5.5.2 Generated Assembly

Disassembly of section .text:

```

00000000 <_start>: 0: 00001117      auipc sp,0x1 4: 20010113      addi sp,sp,512
# 1200 <_stack_top> 8: 008000ef      jal 10

0000000c : c: 0000006f      j c

00000010 : 10: ff010113      addi sp,sp,-16 14: 00a00793      li a5,10 18:
00f12223      sw a5,4(sp) 1c: 01400793      li a5,20 20: 00f12423      sw
a5,8(sp) 24: 00412783      lw a5,4(sp) 28: 00812703      lw a4,8(sp) 2c:
00e787b3      add a5,a5,a4 30: 00f12623      sw a5,12(sp) 34: 00c12503
lw a0,12(sp) 38: 01010113      addi sp,sp,16 3c: 00008067      ret

```

5.5.3 Expected Program Execution

Step	Operation	Expected Result
1	Store variable a	Memory = 10
2	Store variable b	Memory = 20
3	Load a and b	Registers loaded correctly
4	Perform addition	Result = 30
5	Store variable c	Memory = 30
6	Return value	a0 = 30

5.5.4 RTL Verification

riscv64-unknown-elf-nm gcc_files/tst.elf

00001000 B _stack_bottom

00001200 B _stack_top

The generated assembly for the `main()` function begins with:

```
addi sp, sp, -16
```

Initial SP = 0x1200

SP = 0x1200 - 0x10

SP = 0x11F0

Therefore, the memory addresses of the variables are calculated as:

Variable	Calculation	Memory Address
a	$0x11F0 + 4$	0x11F4
b	$0x11F0 + 8$	0x11F8
c	$0x11F0 + 12$	0x11FC

So, in tb we add this :-

```
always @(cpu.dmem.mem[16'h1FC]) begin
```

```
$display("Time=%0t A=%0d B=%0d C=%0d",
```

```
$time,
```

```
cpu.dmem.mem[16'h1F4],
```

```
cpu.dmem.mem[16'h1F8],
```

```
cpu.dmem.mem[16'h1FC]);
```

```
end
```

Result :

```
Running VERILATOR....
```

```
Time=0 A=0 B=0 C=0
```

```
Time=175000 A=10 B=20 C=30
```

```
- ./tb/tb.v:25: Verilog $finish
```

Memory Location	Variable	Expected Value	Observed Value	Status
0x11F4	a	10	10	PASS
0x11F8	b	20	20	PASS
0x11FC	c	30	30	PASS

Chapter 6: Phase IV – Architectural Verification using Spike ISA Simulator

6.1 Introduction

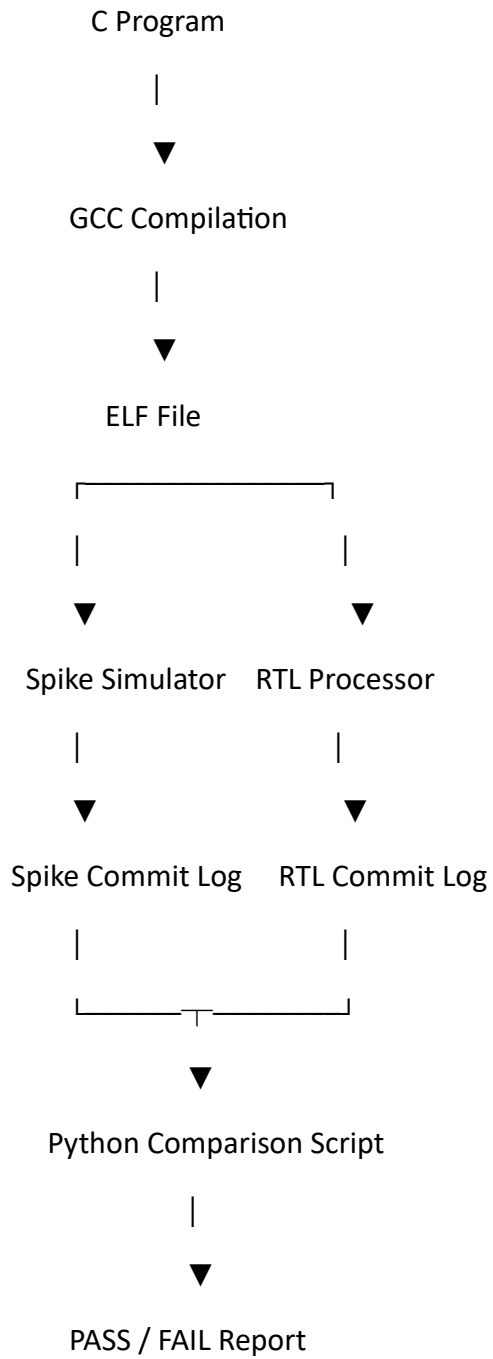
After verifying the processor using directed instruction tests and compiled C programs, the next objective was to validate the processor against a trusted architectural reference model. Manual verification becomes increasingly difficult for larger programs since hundreds or thousands of instructions may execute during a single simulation. Therefore, the Spike RISC-V ISA Simulator was integrated into the verification flow to serve as the golden reference model.

In this phase, the same executable program was run on both Spike and the RTL processor. Execution logs generated by both environments were automatically compared using a Python-based log comparison framework. This enabled instruction-by-instruction verification of the processor and significantly improved confidence in the architectural correctness of the implementation.

6.2 Why Spike?

Spike is the official RISC-V ISA reference simulator developed to model the architectural behavior of RISC-V processors. Since Spike executes instructions according to the RISC-V ISA specification, it serves as a golden reference for validating custom processor implementations. Comparing RTL execution against Spike ensures that every committed instruction produces the expected architectural state.

6.3 Verification Flow



6.4 Spike Commands

1. For generating tst_spike.elf using link_spike.ld:

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -O2 -T gcc_files/link_spike.ld -nostdlib -nostartfiles gcc_files/crt0.S gcc_files/tst.c -o gcc_files/tst_spike.elf
```

2. To run tst_spike.elf on spike and generating log file:

```
spike -l --log-commits --isa=rv32i --instructions=500 gcc_files/tst_spike.elf > spike_commit.log 2>&1
```

3. Command for comapring logs :

```
python3 comapre_logs.py
```

6.5 Python Comparison Script

The Python script performs four major functions.

Function	Purpose
<code>parse_rtl()</code>	Reads the RTL log and extracts register and memory write operations.
<code>parse_spike()</code>	Reads the Spike commit log and extracts the corresponding architectural commits.
<code>equivalent()</code>	Compares the commit type, PC, destination register/memory address, and written data.
Main Comparison Loop	Sequentially compares all commits and reports either PASS or the first mismatch.

6.3 Verification Example

The following example demonstrates the Spike-based architectural verification methodology used to validate the processor.

6.3.1 Test Program

```
int main()
```

```
{
```

```

volatile int a = 10;

volatile int b = 20;

volatile int c = a + b;

return c;
}

```

6.3.2 RTL Commit Log

The RTL processor generated the following architectural commits during execution.

RTL commits

TYPE	PC	RD/ADDR	DATA

reg	00000000	x2	00001000
reg	00000004	x2	00001200
reg	00000008	x1	0000000c
reg	00000010	x2	000011f0
reg	00000014	x15	0000000a
mem	00000018	000011f4	0000000a
reg	0000001c	x15	00000014
mem	00000020	000011f8	00000014
reg	00000024	x15	0000000a
reg	00000028	x14	00000014
reg	0000002c	x15	0000001e
mem	00000030	000011fc	0000001e
reg	00000034	x10	0000001e
reg	00000038	x2	00001200

6.3.3 Spike Commit Log after python script

The same executable was executed on the Spike ISA simulator, producing the following architectural commit log.

Spike commits

TYPE	PC	RD/ADDR	DATA

reg	00000000	x2	00001000
reg	00000004	x2	00001200
reg	00000008	x1	0000000c
reg	00000010	x2	000011f0
reg	00000014	x15	0000000a
mem	00000018	000011f4	0000000a
reg	0000001c	x15	00000014
mem	00000020	000011f8	00000014
reg	00000024	x15	0000000a
reg	00000028	x14	00000014
reg	0000002c	x15	0000001e
mem	00000030	000011fc	0000001e
reg	00000034	x10	0000001e
reg	00000038	x2	00001200

6.3.4 Log Comparison Result

The generated logs were compared using the developed Python comparison framework.

=====

CHECKING

=====

PASS

RTL and Spike logs match

6.6 Architectural Verification Results

To validate the processor under a wide range of execution scenarios, multiple directed C programs were executed on both the RTL processor and the Spike ISA simulator. The execution logs generated by both environments were compared using the developed Python-based comparison framework. The successful comparison of all programs demonstrates the architectural correctness of the implemented RV32I processor.

Test No.	Test Program	Features Verified	Verification Method	Status
1	Fibonacci Sequence	Loops, arithmetic operations, RAW hazards, branches	RTL vs Spike	PASS
2	Bubble Sort	Nested loops, array accesses, loads/stores, branches	RTL vs Spike	PASS
3	Comprehensive ALU Test	ADD, SUB, AND, OR, XOR, immediate instructions	RTL vs Spike	PASS
4	Signed Arithmetic Test	Signed arithmetic, negative numbers, branch instructions	RTL vs Spike	PASS
5	Simple Addition	Basic ALU datapath verification	RTL vs Spike	PASS
6	Memory Store Test	Store instructions and data memory operations	RTL vs Spike	PASS
7	Constant Arithmetic Test	Immediate arithmetic instructions	RTL vs Spike	PASS
8	Branch Decision Test	BLT and conditional branch verification	RTL vs Spike	PASS
9	RAW Dependency Chain	Forwarding logic and RAW hazard handling	RTL vs Spike	PASS

Test No.	Test Program	Features Verified	Verification Method	Status
10	Factorial Program	Nested loops and arithmetic-intensive execution	RTL vs Spike	PASS
11	Load-Use Hazard Test	Hazard detection and pipeline stall generation	RTL vs Spike	PASS
12	RAW Hazard Stress Test	Multiple dependent instructions and forwarding	RTL vs Spike	PASS
13	Forwarding Test	EX/MEM and MEM/WB forwarding verification	RTL vs Spike	PASS
14	Combined Hazard Test	Load-use, RAW and branch hazards	RTL vs Spike	PASS
15	Multiple Load/Store Test	Data memory read/write verification	RTL vs Spike	PASS
16	Full Pipeline Hazard Test	Load, store, RAW and control hazards	RTL vs Spike	PASS
17	Prime Number Detection	Branch-intensive execution and nested loops	RTL vs Spike	PASS
18	Complex Dependency Chain	Multiple forwarding paths and dependency chains	RTL vs Spike	PASS
19	Bubble Sort Stress Test	Memory accesses, nested loops and branches	RTL vs Spike	PASS
20	Matrix Transpose	Two-dimensional array accesses and memory operations	RTL vs Spike	PASS
21	Selection Sort	Sorting algorithm with nested loops and memory accesses	RTL vs Spike	PASS

The successful execution and architectural comparison of the above test programs confirmed the correct implementation of arithmetic, logical, memory, control-flow, forwarding, hazard detection, and pipeline control mechanisms in the developed RV32I processor. No architectural

mismatches were observed between the RTL processor and the Spike ISA simulator for the executed test suite.

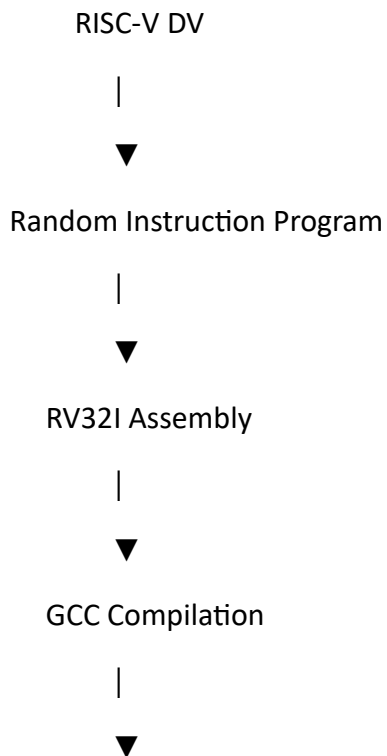
Chapter 7: Phase V – Constrained Random Verification using RISC-V DV

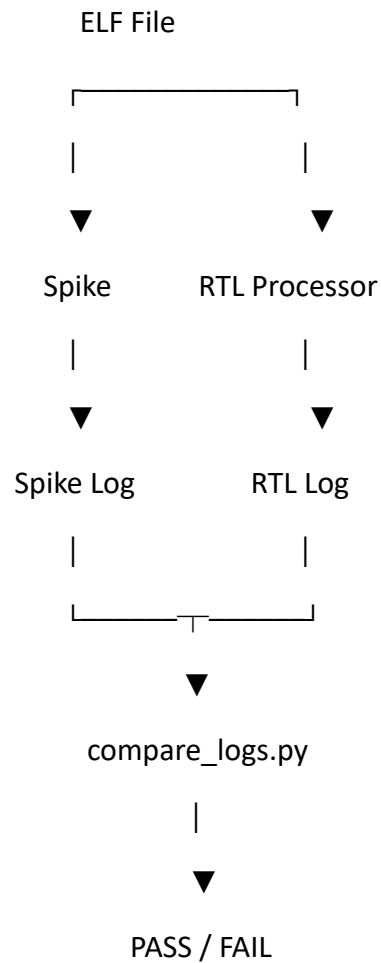
7.1 Introduction

After successful directed verification and architectural validation using the Spike ISA simulator, the next stage of processor verification focused on constrained-random testing using the RISC-V DV framework. While directed test programs verify known execution scenarios, they cannot cover every possible instruction sequence or corner case. RISC-V DV automatically generates random instruction streams that exercise different combinations of instructions, data dependencies, pipeline hazards, and control-flow patterns, providing significantly higher verification coverage.

In this phase, the generated random programs were executed on both the RTL processor and the Spike ISA simulator. Their execution logs were then compared using the previously developed Python-based comparison framework to identify any architectural mismatches.

7.2 RISC-V DV Verification Flow





7.3 Verification Procedure

The following verification procedure was adopted during random testing.

Step	Description
1	Generate a random RV32I program using RISC-V DV.
2	Remove unsupported instructions (CSR, privilege instructions, etc.) to obtain a pure RV32I program.
3	Compile the generated assembly using the GNU RISC-V toolchain.
4	Execute the generated program on both the RTL processor and Spike.
5	Generate architectural commit logs.
6	Compare the execution logs using the Python comparison framework.

Step	Description
7	Debug any mismatch and repeat the verification process.

7.4 RISC-V DV Verification Workflow

The complete verification flow adopted for validating randomly generated RISC-V programs is illustrated below.

Step 1: Open the RISC-V DV Environment

Navigate to the RISC-V DV workspace and activate the Python virtual environment.

```
cd ~/riscv_core_dv_workspace/riscv-dv
source .venv/bin/activate
```

Step 2: Generate a Random Test Program

Generate a random arithmetic test using the RISC-V DV framework.

```
python run.py \
--test riscv_arithmetic_basic_test \
--iterations 1 \
--target rv32i \
--isa rv32i \
--mabi ilp32 \
--simulator pyflow \
--steps gen
```

Output:

- riscv_arithmetic_basic_test_0.S

Step 3: Convert to a Pure RV32I Program

The generated assembly contains CSR and privileged instructions that are not supported by the developed processor. Therefore, only RV32I instructions are extracted.

```
python3 extract_rv32i.py \
```

```
~/riscv_core_dv_workspace/riscv-dv/out_XXXXX/asm_test/riscv_arithmetic_basic_test_0.S \
rv32i_only.S
```

Output:

- rv32i_only.S

Step 4: Compile the Assembly Program

Compile the extracted RV32I program using the GNU RISC-V toolchain.

```
riscv64-unknown-elf-gcc \
-march=rv32i \
-mabi=ilp32 \
-nostdlib \
-T link.ld \
rv32i_only.S \
-o test.elf
```

Output:

- test.elf

Step 5: Generate Instruction Memory

Convert the executable into the instruction memory format required by the RTL processor.

```
echo "@00000000" > imem.mem
```

```
riscv64-unknown-elf-objcopy \
-O binary \
test.elf test.bin
```

```
hexdump -v -e '1/4 "%08X\n"' \
test.bin >> imem.mem
```

Output:

- `imem.mem`

Step 6: Generate Data Memory

Extract the initialized data section.

```
riscv64-unknown-elf-objcopy \
```

```
-O binary \
```

```
--only-section=.data \
```

```
test.elf dmem.bin
```

```
hexdump -v -e '1/4 "%02X\n"' \
```

```
dmem.bin > dmem.mem
```

Output:

- `dmem.mem`

Step 7: Execute on RTL Processor

Compile and execute the RTL processor using Verilator.

```
cd verilator
```

```
make clean
```

```
make verilate
```

```
./obj_dir/Vtb
```

Output:

- RTL execution
- `rtl.log`

Step 8: Execute on Spike

Run the same executable on the Spike ISA simulator.

```
spike \  
-l \  
--log-commits \  
--isa=rv32i \  
--instructions=500 \  
test_spike.elf \  
> spike_commit.log 2>&1
```

Output:

- spike_commit.log

Step 9: Compare RTL and Spike Logs

Execute the Python comparison framework.

```
python3 compare_logs.py
```

Output:

```
=====
```

```
CHECKING
```

```
=====
```

```
PASS
```

RTL and Spike logs match

7.5 RISC-V DV Test Status

The RISC-V DV framework provides a comprehensive collection of constrained-random test generators for validating different aspects of a RISC-V processor. Based on the features currently implemented in the developed RV32I processor, the following tests have been selected for verification.

Test Name	Purpose	Current Status
<code>riscv_arithmetic_basic_test</code>	Random arithmetic and logical instruction verification	Completed
<code>riscv_jump_stress_test</code>	Back-to-back JAL instruction verification	Completed
<code>riscv_rand_instr_test</code>	Random instruction stream with data dependencies and hazards	In Progress
<code>riscv_loop_test</code>	Random loop generation and control-flow verification	Completed
<code>riscv_rand_jump_test</code>	Random sub-program jumps and control transfer verification	In Progress
<code>riscv_mmu_stress_test</code>	Random load/store stress patterns	In Progress

The remaining tests provided by the RISC-V DV framework, such as CSR verification, interrupt handling, debug mode verification, illegal instruction generation, and unaligned memory access tests, require architectural features that are not implemented in the current RV32I processor and are therefore outside the scope of this work.

7.6 Overall Verification Summary

The verification of the developed five-stage RV32I processor was carried out through a structured and progressive methodology. The verification process began with directed instruction-level testing and gradually advanced toward architectural validation using the Spike ISA simulator and constrained-random verification using the RISC-V DV framework. Each verification phase increased the complexity of the executed programs, enabling systematic debugging and validation of the processor.

The overall verification results are summarized below.

Verification Category	Description	Status
Instruction-Level Verification	Manual verification of all RV32I instructions using machine code and RTL waveforms	PASS

Verification Category	Description	Status
Arithmetic Instructions	ADD, SUB, SLT, SLTU and immediate arithmetic instructions	PASS
Logical Instructions	AND, OR, XOR and immediate logical instructions	PASS
Shift Instructions	SLL, SRL, SRA and immediate shift instructions	PASS
Upper Immediate Instructions	LUI and AUIPC	PASS
Load and Store Instructions	Byte, Half-word and Word memory operations	PASS
Branch Instructions	BEQ, BNE, BLT, BGE, BLTU and BGEU	PASS
Jump Instructions	JAL and JALR	PASS
Data Hazard Verification	RAW hazards, forwarding paths and dependency chains	PASS
Load-Use Hazard Verification	Hazard detection and pipeline stall generation	PASS
Double Load-Use Hazard	Register file read-before-write priority verification	PASS
Control Hazard Verification	Branch flush and program counter update	PASS
GNU Toolchain Verification	Compilation and execution of C programs on the RTL processor	PASS
Software Workload Verification	Multiple C programs including sorting, Fibonacci, factorial, matrix operations and memory tests	PASS
Spike Architectural Verification	Instruction-by-instruction comparison of RTL and Spike commit logs	PASS
Python Log Comparison Framework	Automated comparison of register and memory commits	PASS
RISC-V DV Arithmetic Test	Random arithmetic and logical instruction verification	PASS

Verification Category	Description	Status
RISC-V DV Jump Stress Test	Random jump and JAL verification	PASS
RISC-V DV Loop Test	Random loop and control-flow verification	PASS
RISC-V DV Random Instruction Test	Random instruction sequences with hazards	In Progress
RISC-V DV Random Jump Test	Random sub-program jump verification	In Progress
RISC-V DV MMU Stress Test	Random load/store stress verification	In Progress